

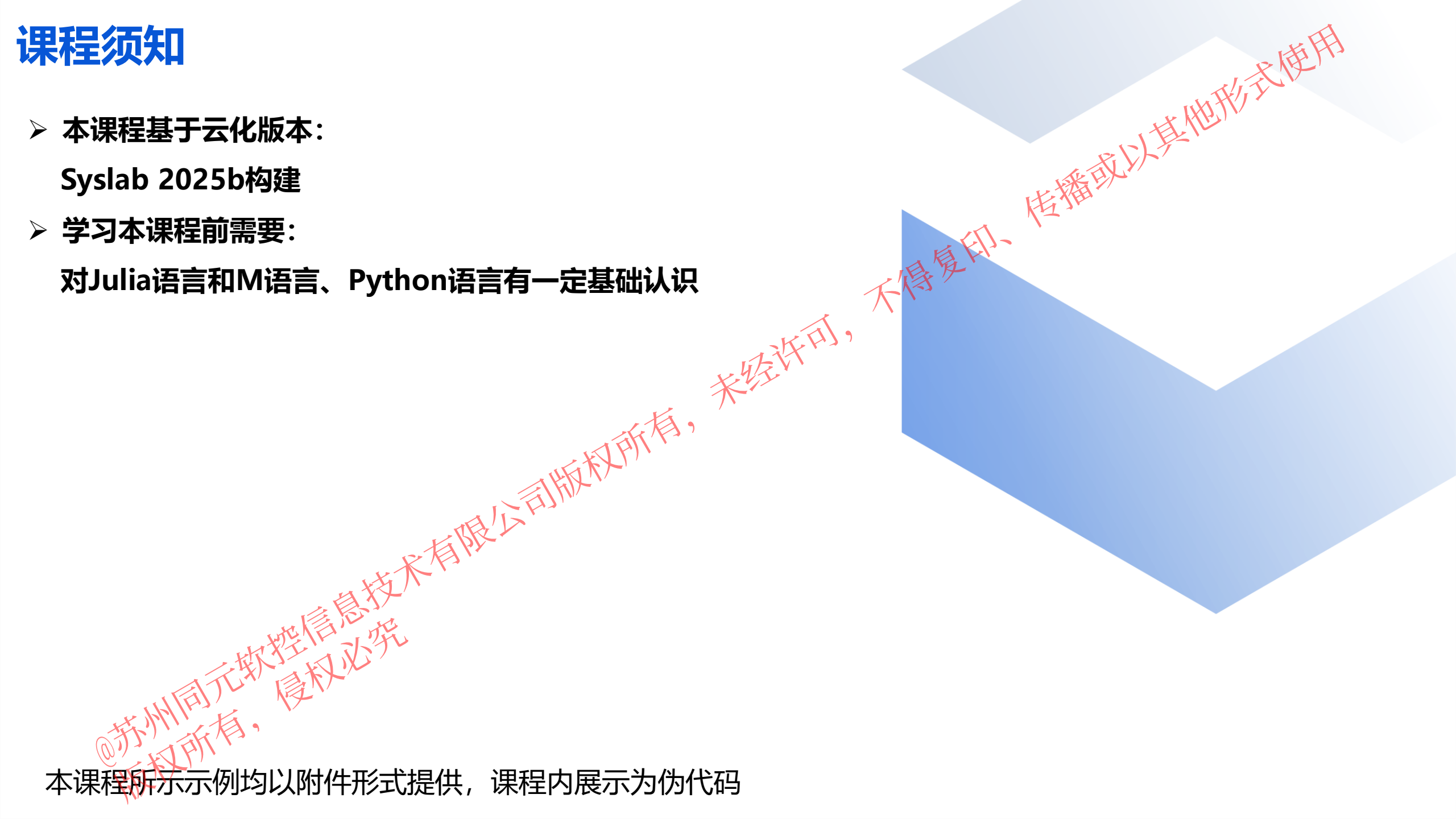
Syslab M语言兼容

汇报人：王慧敏



课程须知

- 本课程基于云化版本：
Syslab 2025b构建
- 学习本课程前需要：
对Julia语言和M语言、Python语言有一定基础认识



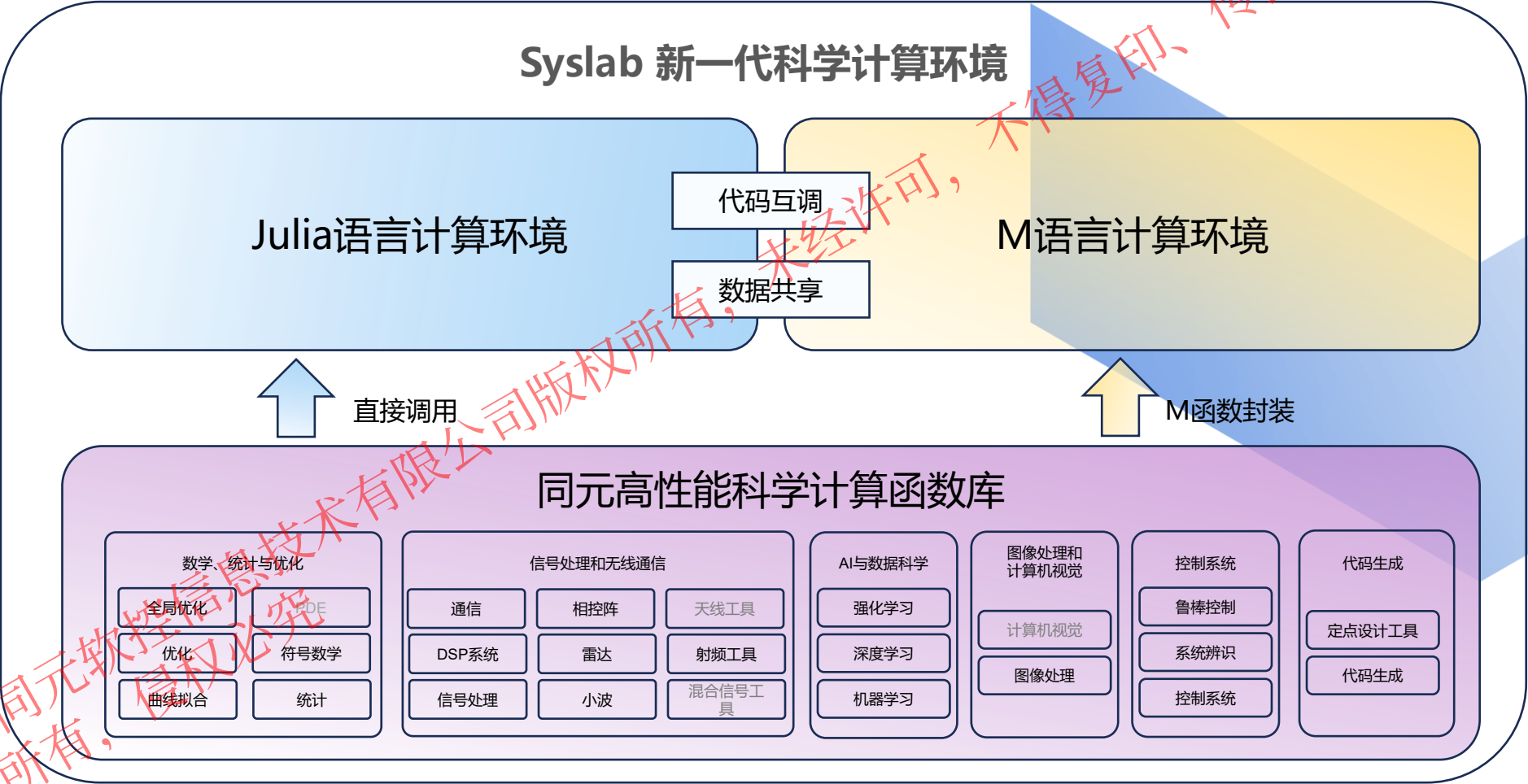
不得复印、传播或以其他方式使用

@苏州同元软控信息技术有限公司版权所有，侵权必究

本课程所示示例均以附件形式提供，课程内展示为伪代码

Syslab与M语言计算环境

Syslab 是新一代科学计算环境，以Julia语言为主，同时兼容M语言，支持 Julia 与M、Python等编程语言的相互调用

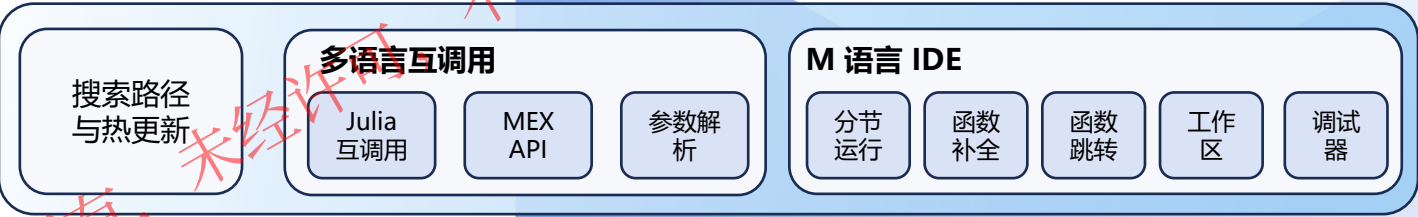


M语言计算环境-工具定位

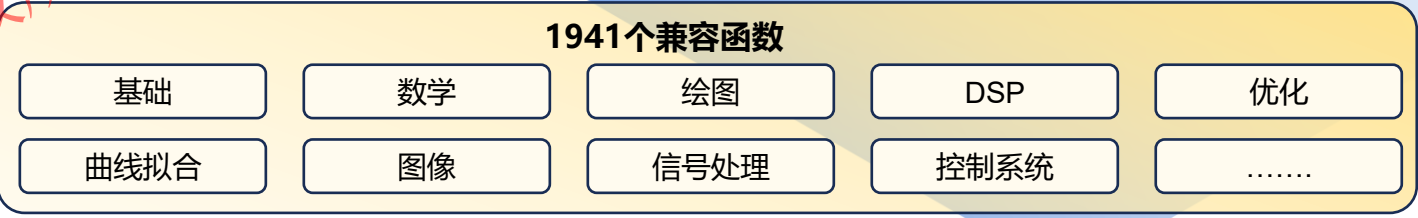
TyMLang（M语言兼容）是同元软控实现的一款类 MATLAB/Octave 编程语言，旨在使熟悉 MATLAB/Octave的用户能轻松介入 Julia 编程语言及其先进的科学计算生态，同时仍能使用 MATLAB/Octave 用户熟悉的语法和界面操作。

- **MATLAB语言级兼容**：系统性的原生支持MATLAB/M语言的核心语法、程序行为和机制。
- **内置常用函数**：内置提供基础、数学、图形、控制系统、信号处理、通信等领域常用函数。
- **M与Julia语言互调用**：用户可以接入Julia生态，能够调用Julia的诸多科学计算库。
- **复用存量M代码**：无需安装MATLAB，无需修改源码，即可实现存量M代码的兼容运行。

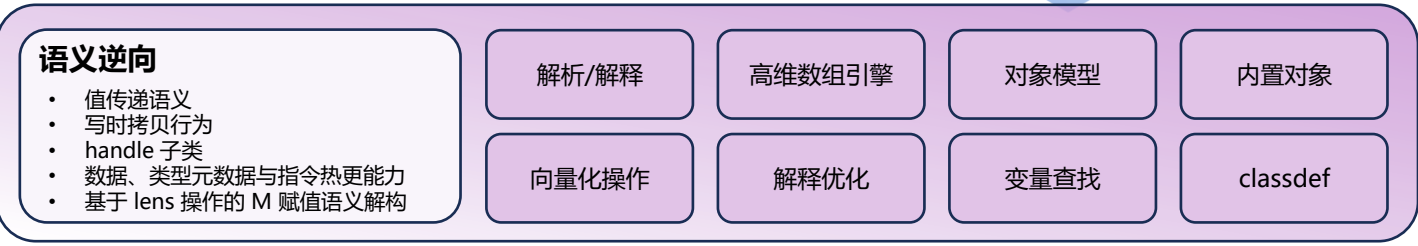
交互层



函数层



内核层



@苏州同元软控信息技术有限公司
版权所有，侵权必究

目录 ➔



M兼容基础设置



M脚本运行与调试



M兼容高阶功能

PART 01 ➔

M兼容基础设置

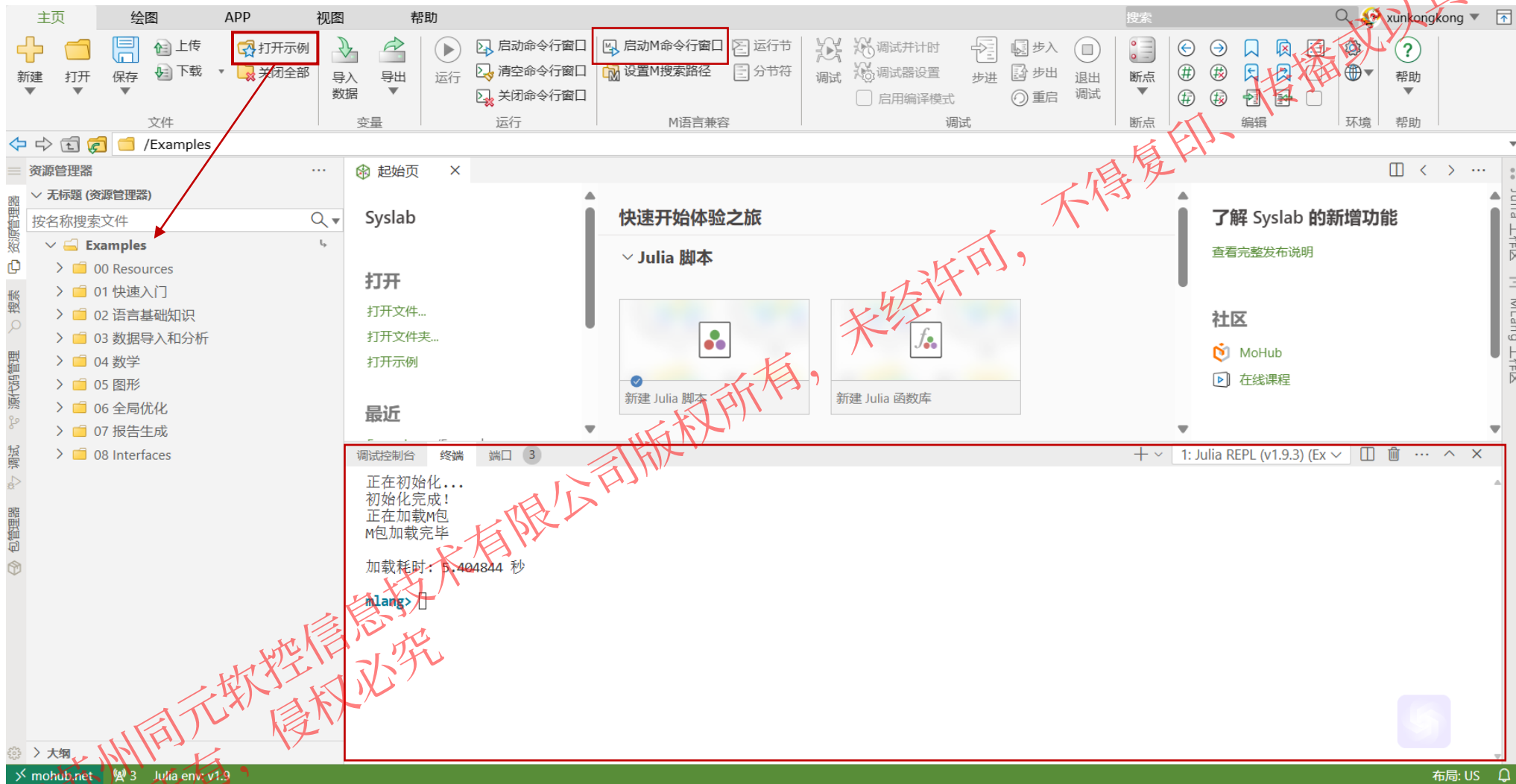
@苏州同元软控信息技术有限公司
版权所有，侵权必究

未经许可，不得复印、传播或以其他方式使用

01

1.1 启动

单击主页面板**启动 M 命令行窗口**按钮即可启动 M 语言计算环境命令行

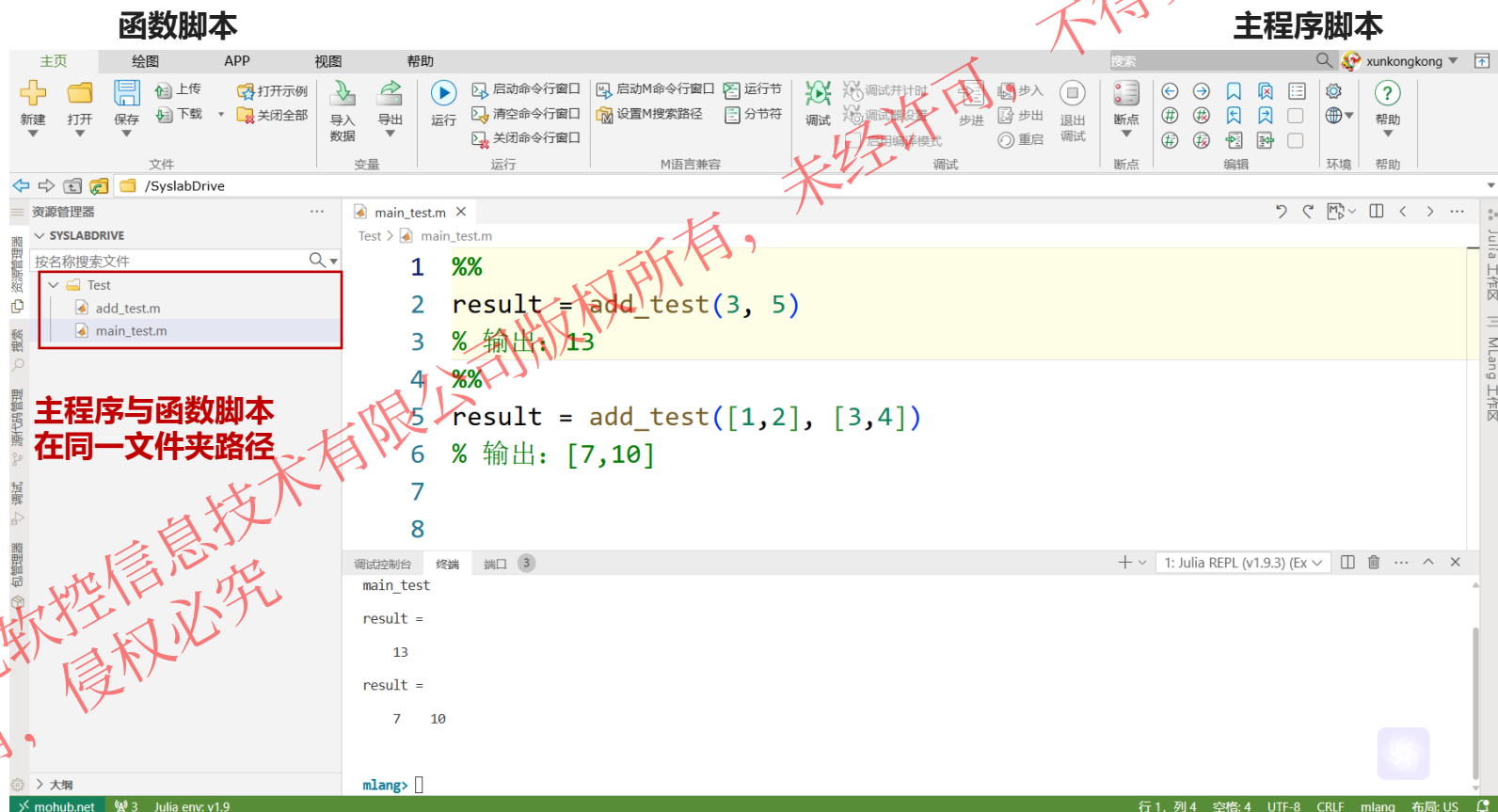


1.2 设置路径

与 MATLAB 一致，资源管理器打开至主程序与函数脚本所在文件夹，即可直接实现函数调用

```
% 定义一个加法函数
function sum = add_test(a, b)
    sum = a + 2*b;
end
```

```
result = add_test(3, 5)
% 输出: 13
result = add_test([1,2], [3,4])
% 输出: [7,10]
```



1.2 设置路径

M 语言计算环境支持**搜索路径**功能：只要 M 函数文件位于搜索路径中，可按文件名直接调用相应 M 函数。除内置函数库外，还支持**将自定义函数路径添加到搜索路径**

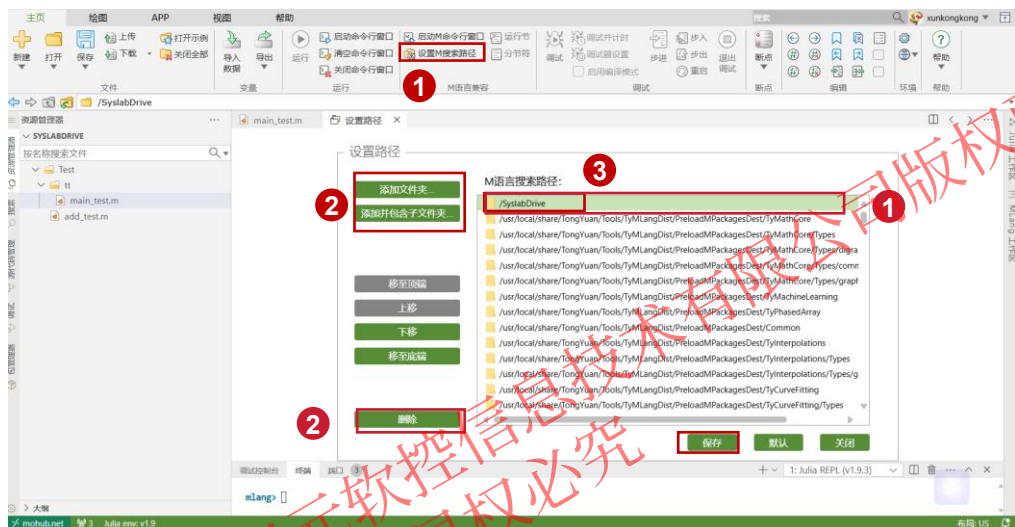
可视化操作

添加路径操作步骤：

1. 单击设置 M 搜索路径
2. 单击添加文件夹或添加并包含子文件夹
3. 选择目标文件夹，然后保存

删除路径操作步骤：

1. 选择需要删除的路径选项后单击删除，然后保存



说明：

- 添加文件夹：仅添加当前文件夹
- 添加并包含子文件夹：添加当前文件夹，并包含所有子文件夹

文本操作

添加路径操作步骤：

1. 使用 addpath 命令添加路径

```
mlang>addpath('SyslabDrive/Test')
```

删除路径操作步骤：

1. 使用 rmpath 命令删除路径

```
mlang>rmpath('SyslabDrive/Test')
```

```
mlang> addpath('/SyslabDrive/Test')
```

添加路径

```
mlang> rmpath('/SyslabDrive/Test')
```

删除路径

说明：

- addpath 仅添加当前路径下的 M 文件，不添加子文件夹下的路径
- 清空路径之后重启 Syslab 命令行，搜索路径即可恢复到默认状态

02

PART 02 ➔

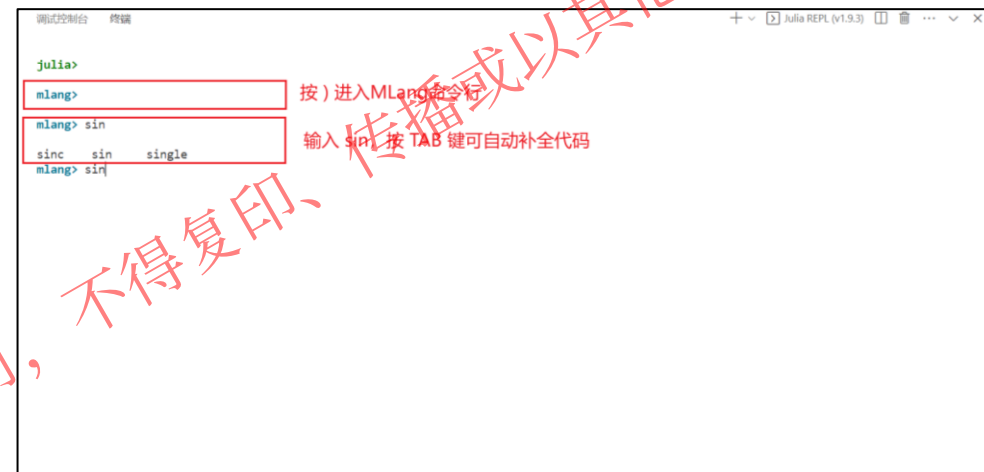
M脚本运行与调试

@苏州同元软控信息技术有限公司版权所有，侵权必究

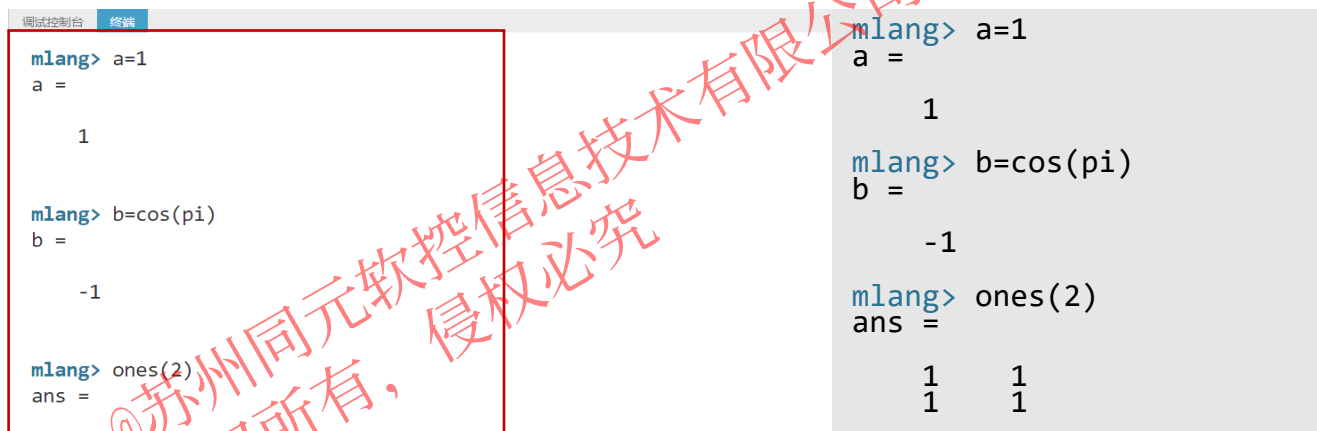
未经许可，不得复印、传播或以其他方式使用

2.1 MLang命令行

- MLang快捷键切换
 - 当命令行窗口显示mlang时，表示此时在M语言命令行
 - 按 **Backspace** ，回到Julia命令行
 - 若想再次开启mlang命令行，可以点击[M语言兼容] 处的 [启动M命令行窗口] 按钮或者按右括号) 重新进入 mLang 命令行
- M_REPL自动补全
 - 与Julia 命令行中相同，在 MLang 命令行下可按 TAB 键自动补全代码



使用 MLang命令行时，和 M 语言语法一致，可以创建变量、调用函数等



- 如果未指定输出变量，M 语言计算环境将使用变量 ans（answer 的缩写形式）来存储计算结果
- 如果语句以分号结束，M 语言计算环境会执行计算，但不在命令行窗口中显示输出
- 按 ↑ 键和按 ↓ 键可以重新调用以前的命令。此外，在空白命令行中键入命令的前几个字符后按下箭头键，可获取相同行首的历史命令。例如：要重新调用命令 b = cos(pi)，请键入 b，然后按 ↑ 键。

2.1 Mlang编辑器

提供M代码编辑器，主要支持对M代码文本的编辑功能

包括语法高亮、

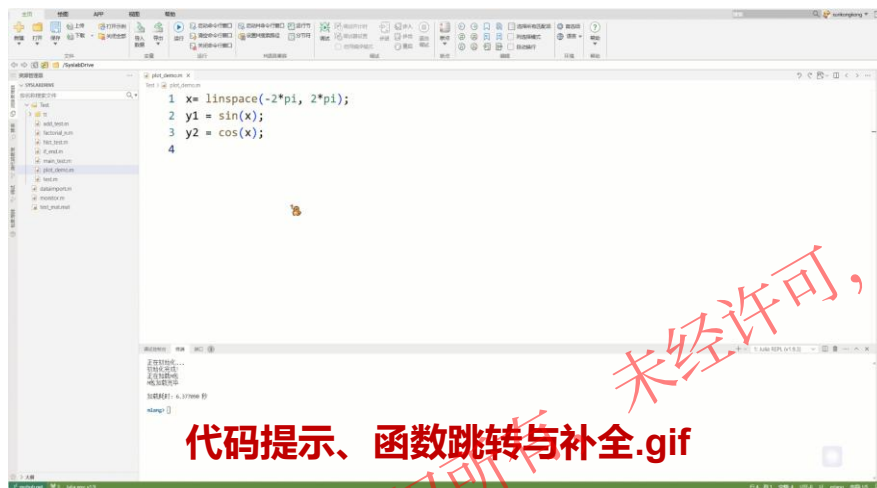
代码提示、函数跳转与补全

格式化、

重命名、

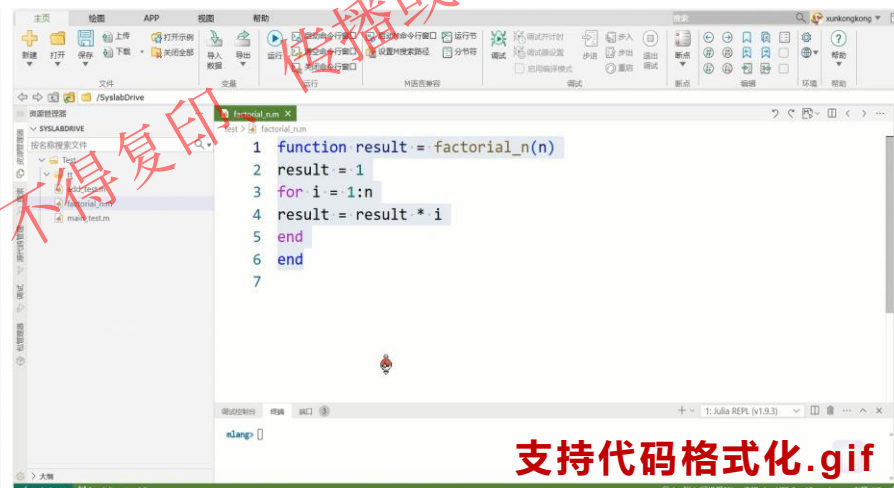
插入分节符、运行节

块注释和行注释等功能。



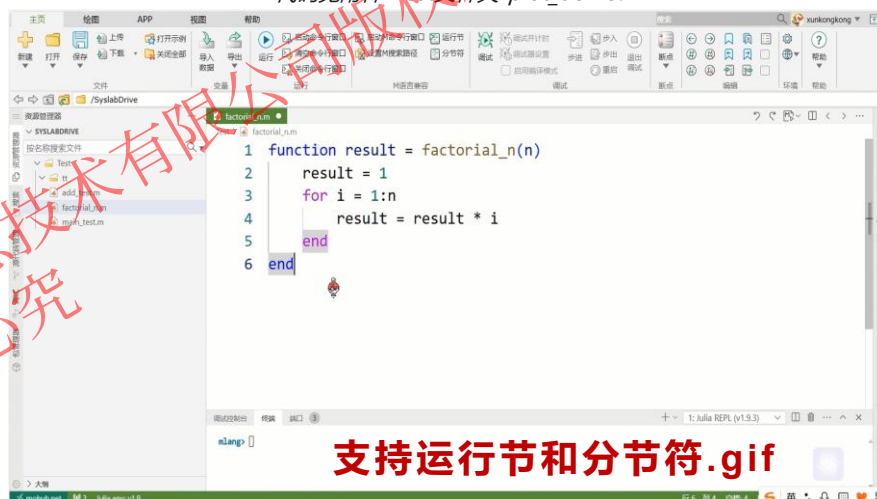
代码提示、函数跳转与补全.gif

代码见附件-Test文件夹-plot_demo.m



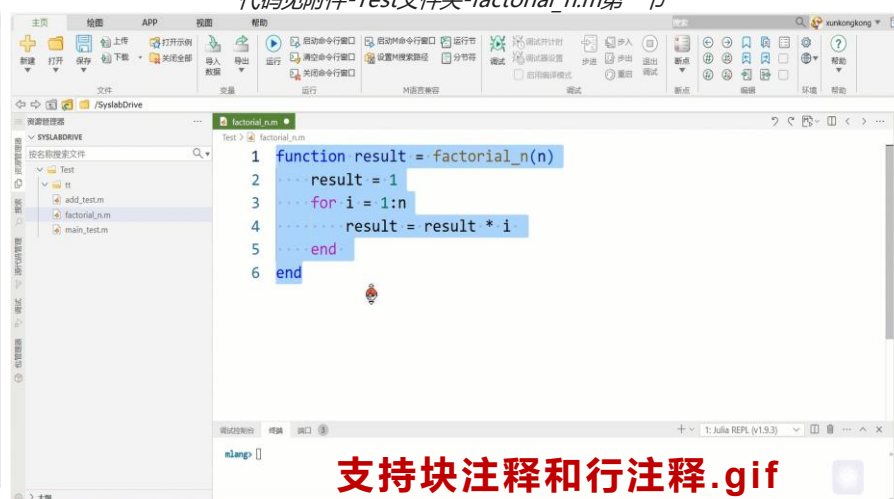
支持代码格式化.gif

代码见附件-Test文件夹-factorial_n.m第一节



支持运行节和分节符.gif

代码见附件-Test文件夹-factorial_n.m第一节



支持块注释和行注释.gif

```
%%  
function result = factorial_n(n)  
    result = 1  
    for i = 1:n  
        result = result * i  
    end  
end  
%%  
a=1
```

@苏州同元软控信息技术有限公司
版权所有，侵权必究

2.2 MLang工作区

支持对命令行窗口中变量、类型、函数等元素进行集中显示与编辑，支持双击变量打开变量编辑器

The screenshot displays the MLang IDE interface. The top menu bar includes '主页' (Home), '绘图' (Plot), 'APP', '视图' (View), and '帮助' (Help). The '视图' menu is open, showing options like '启动命令行窗口', '启动M命令行窗口', '运行', '清空命令行窗口', '设置M搜索路径', '分节符', '调试', '调试并计时', '调试器设置', '启用编译模式', '步进', '步出', '退出', '断点', '帮助'. The '导出' (Export) option is highlighted. The main workspace shows a file explorer on the left with a folder 'Test' containing files 'add_test.m', 'factorial_n.m', and 'main_test.m'. The central code editor displays a Julia script defining variables for Age, Weight, Height, BloodPressure, and T. The right sidebar shows the 'MLANG 工作区' (MLANG Workspace) with a list of variables: Age, BloodPressure, Height, Smoker, T, and Weight. A context menu is open over the 'Height' variable, showing options like '打开所选内容', '导出 mat 文件...', '复制全名', '删除', '重命名', and various plot functions. The bottom status bar shows 'mohub.net', 'Julia env: v1.9', and 'mlang>'. A terminal window at the bottom displays the output of the script.

注意:

- MLang 工作区和 Julia 工作区数据，可以通过函数进行数据传递，见下节高阶功能-Julia语言调用M函数
- MLang 工作区数据支持通过主页菜单栏导入和导出数据

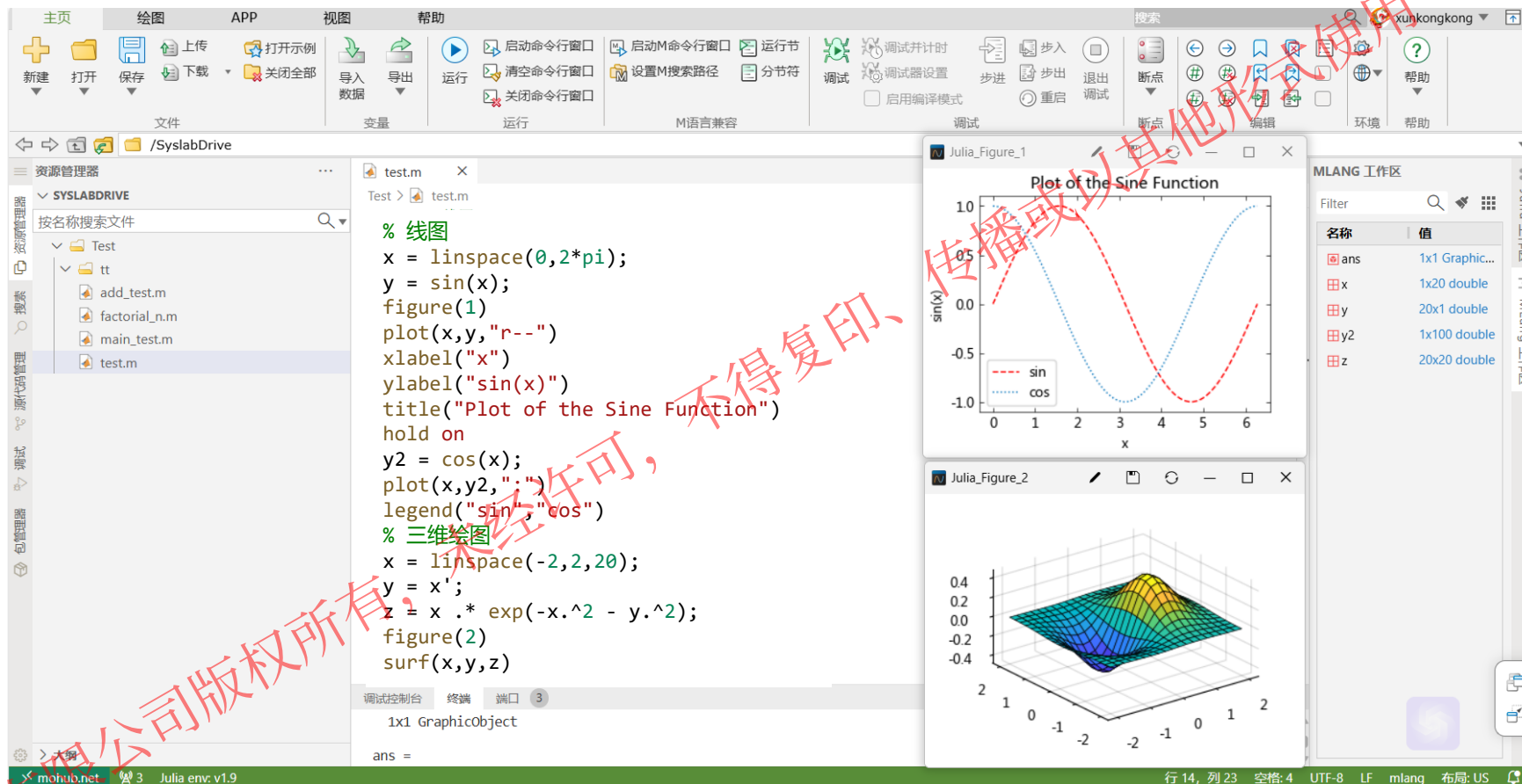
代码见附件-Test文件夹-factorial_n.m第二节

2.3 M脚本运行

一：编写M语言脚本并运行

步骤：

- 创建.m文件，编写M语言脚本，编写时遵循M语言语法
- 单击**运行**按钮，执行该脚本



二：存量代码转移

步骤：

1. 将存量代码文件夹添加到搜索路径，添加方式可参考本材料” 1.2-设置路径”

addpath(“ 存量代码路径”)

2. 单击**运行**按钮

代码选自M软件帮助文档-示例-二维图和三维图，见附件Test文件夹-test.m

2.3 M脚本运行

示例：二维图和三维图绘制

% 线图

```
x = linspace(0,2*pi);  
y = sin(x);  
figure(1)  
plot(x,y,"r--")  
xlabel("x")  
ylabel("sin(x)")  
title("Plot of the Sine Function")
```

hold on

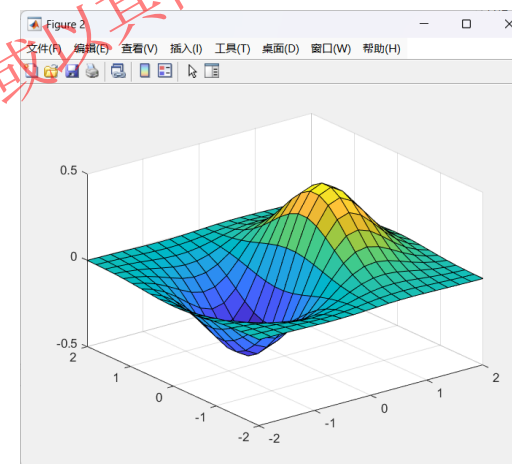
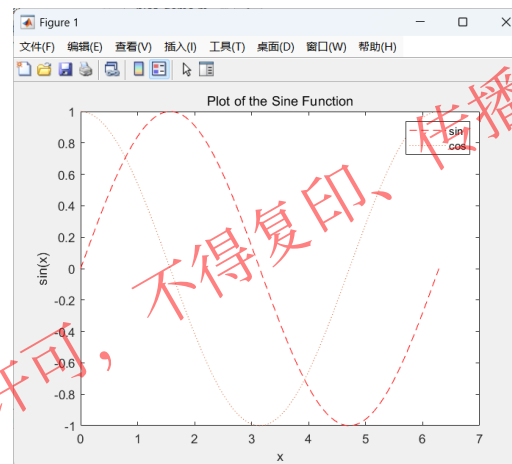
```
y2 = cos(x);  
plot(x,y2,":")  
legend("sin","cos")
```

% 三维绘图

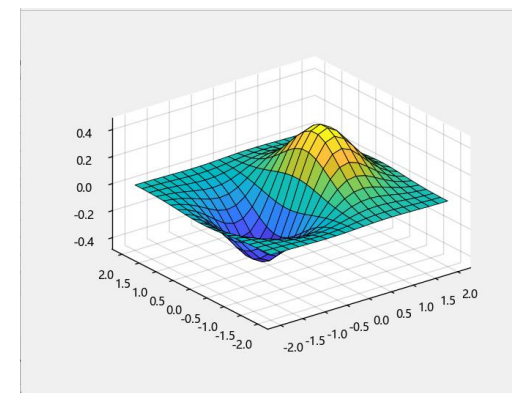
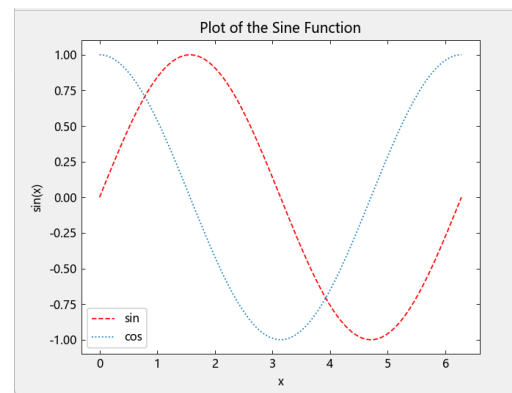
```
x = linspace(-2,2,20);  
y = x';  
z = x .* exp(-x.^2 - y.^2);  
figure(2)  
surf(x,y,z)
```

M脚本

代码选自M软件帮助文档-示例-二维图和三维图，见附件Test文件夹-test.m



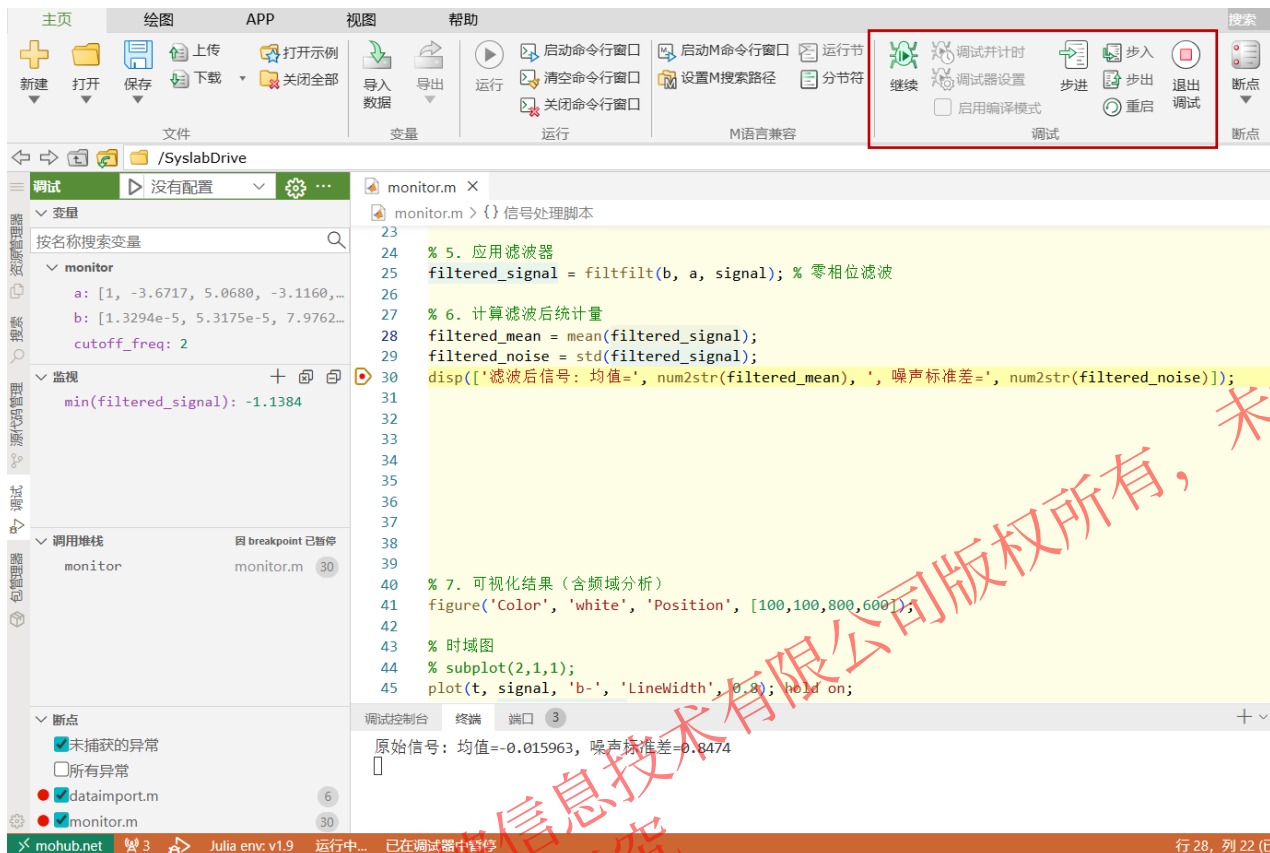
M软件运行结果



Syslab运行结果

2.4 M脚本调试

支持以调试模式运行 M 代码文件，提供**单步调试**、**断点调试**、**查看调用堆栈**、**变量查询操作**等，并提供**交互式调试控制台**



操作步骤:

1. 在M脚本打上断点
2. 点击菜单栏中"启动调试"或F5进入DeBug模式

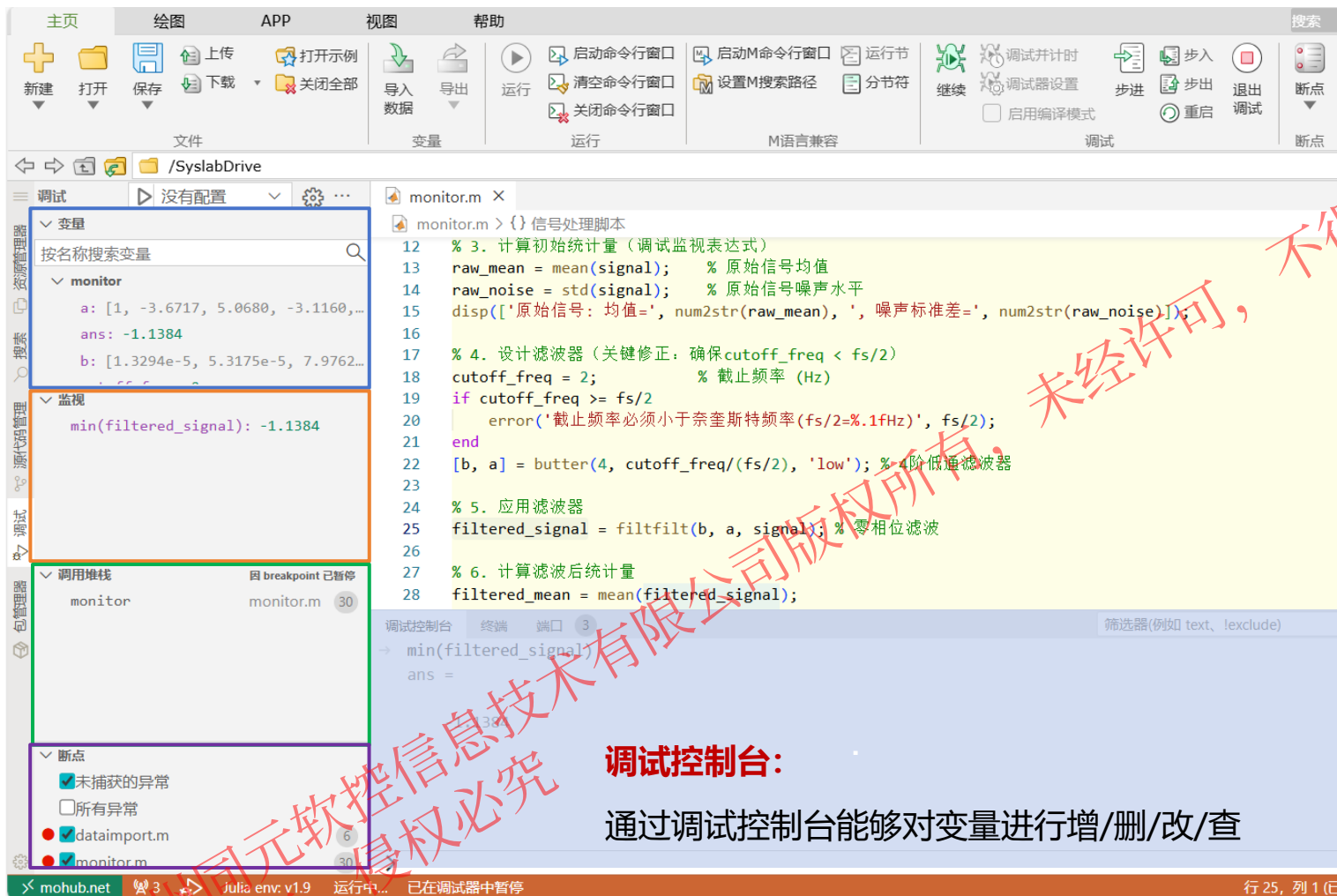
说明:

- **继续(F5):** 继续运行调试
- **步进(F10):** 单步执行遇到子函数时不会进入子函数内，而是将子函数整个执行完再停止
- **步入(F11):** 单步执行遇到子函数就进入并且继续单步执行
- **步出(Shift+F11):** 当单步执行到子函数内时，执行完子函数余下部分，并返回到上一层函数
- **重启(Ctrl+Shift+F5):** 重新启动调试
- **退出调试(Shift+F5):** 停止调试

代码见附件PPT动图演示-调试监视变量功能演示: monitor.m

2.4 M脚本调试

支持以调试模式运行 M 代码文件，提供**单步调试**、**断点调试**、**查看调用堆栈**、**变量查询操作**等，并提供**交互式调试控制台**



说明:

- **变量:** 查看调试过程中的变量值
- **调用堆栈:** 用于了解函数间的调用关系和逻辑
- **断点:** 模型中所有断点的信息

提示:

- 调试控制台目前支持显示基本类型的数据，其他只显示类型和 size

调试控制台:

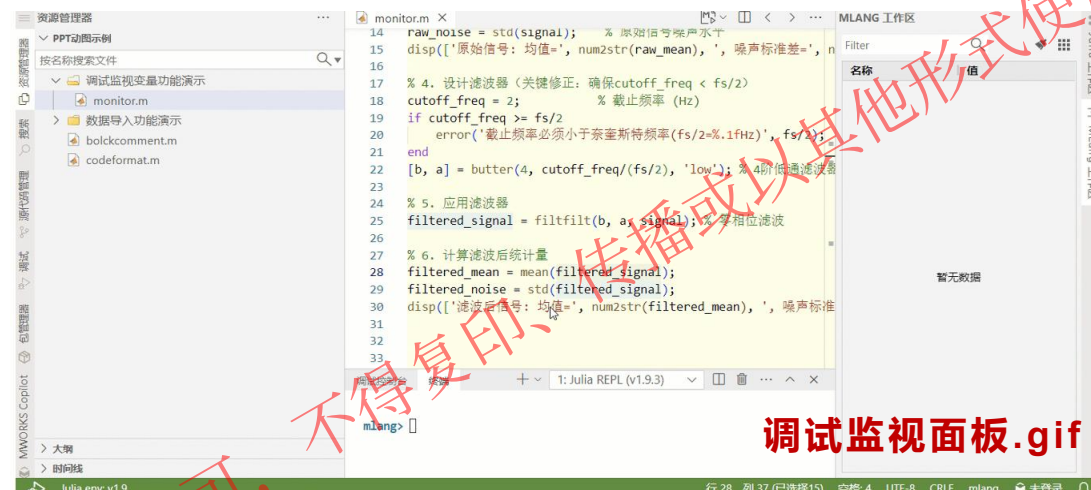
通过调试控制台能够对变量进行增/删/改/查

代码见附件PPT动图演示-调试监视变量功能演示: monitor.m

2.4 M脚本调试

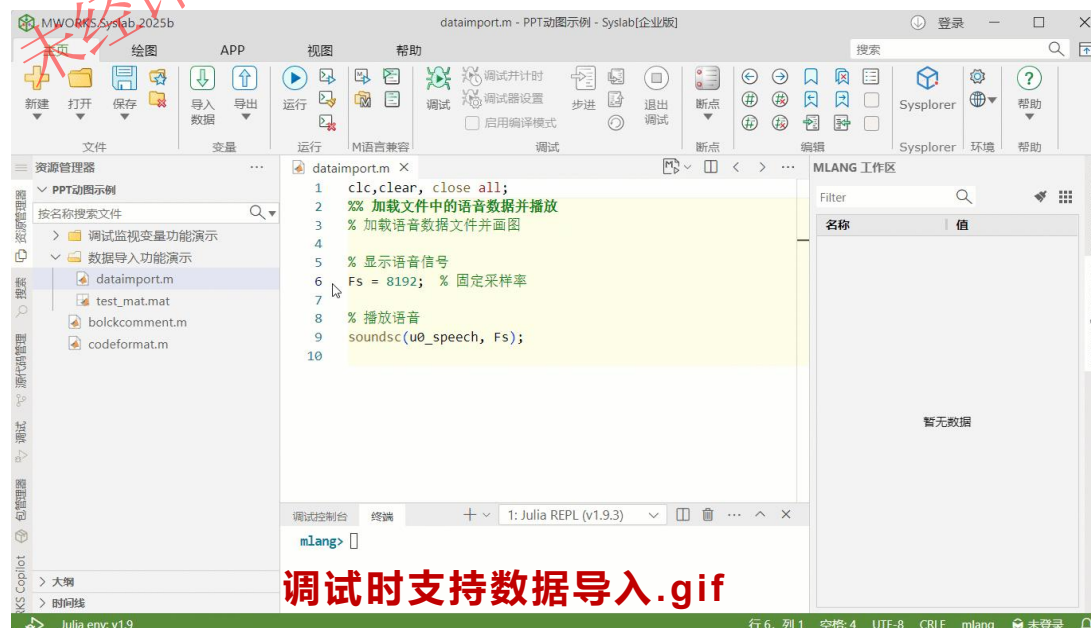
说明:

- 支持变量表格视图查看
- 支持右键变量的导出mat文件、绘图、复制
- 支持调试监视
- 支持调试数据导入



调试监视面板.gif

代码见附件PPT动图演示-调试监视变量功能演示: monitor.m



调试时支持数据导入.gif

代码见附件PPT动图演示-数据导入功能演示: dataimport.m和test_mat

PART 03 ➡

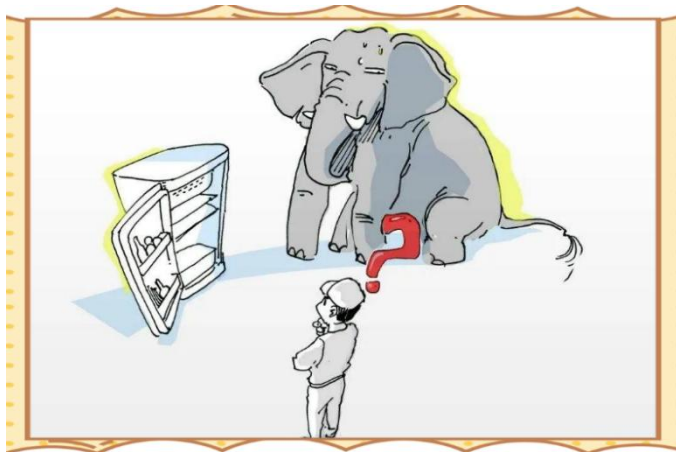
M兼容高阶功能

03

@苏州同元软控信息技术有限公司版权所有，未经许可，不得复印、传播或以其他方式使用
版权所有，侵权必究

3.1 M 语言调用 Julia 函数-将 Julia 函数封装为 M 函数

“把大象装进冰箱拢共分几步？”



“碰到 MLang 未定义和不识别的函数处理拢共分几步？”

```
demo

tymlang.VarNotFound: 变量或函数 latcfilt 无法识别
at  \demo.m:11:7, demo
at  :1:91, demo
at  :1:91,
```



三步:

步骤	要点
1. M 类型转为 Julia 类型	M 类型如何转为 Julia 类型? 有什么规则?
2. 调用 Julia 函数	如何调用已有的 Julia 库函数? Julia 函数返回值为 Julia 类型
3. Julia 类型转为 M 类型	Julia 类型如何转为 M 类型? 有什么规则?

3.1 M 语言调用 Julia 函数-第一步: M 类型转为 Julia 类型 (ToJulia)

Julia 函数用法

% 将 M 类型转为 Julia 类型

```
jv = Julia(o)
```

```
jv = Julia(o, jtype)
```

jv = Julia(o): 默认转换, o 为 M 类型对象, 将 o 转为 Julia 类型对象

%jv = Julia(o): 默认转换, 将转为对应类型的 Array

% Matlab中数值类型默认为 double 类型

```
jv1_1 = Julia(rand(3,1))
```

```
jcall('typeof', jv1_1)
```

```
jv1_2 = Julia(rand(1, 3))
```

```
jcall('typeof', jv1_2)
```

```
jv1_3 = Julia(1)
```

```
jcall('typeof', jv1_3)
```

jv = Julia(o, jtype): jtype 为字符串表示的 Julia 类型, 将 o 转为 类型为 jtype 的 Julia 类型对象

% Julia 的类型比较严格, 对于 Julia 函数来说, 输入参数可能是以下三类: 标量、向量、数组 (不包含向量)

% M 的向量类型, 无论行列向量都可以转为 Julia 的向量 (Vector)

```
jv2_1 = Julia(rand(3,1), 'Vector{Float64}')
```

```
jcall('typeof', jv2_1)
```

```
jv2_2 = Julia(rand(1, 3), 'Vector')
```

```
jcall('typeof', jv2_2)
```

```
jv2_3 = Julia(1, 'Int64')
```

```
jcall('typeof', jv2_3)
```

% Mlang 与 Julia 数据类型对应关系

double -> Float64

single -> Float32

complex -> ComplexF64

string -> String

char -> String

uint8 -> UInt8

uint16 -> UInt16

uint32 -> UInt32

uint64 -> UInt64

int8 -> Int8

int16 -> Int16

int32 -> Int32

int64 -> Int64

struct -> NamedTuple

红色为暂不支持转换类型

3.1 M 语言调用 Julia 函数-第二步：调用 Julia 函数 (jcall)

jcall 函数用法

% 调用 Julia 函数

```
jcall(expr, jv1, ..., jvn)
jcall(__, '-kwargs', kws)
jcall('-return', mtype, __)
jcall('-vector', __)
jcall('-noreturn', __)
```

```
jcall(expr, jv1, ..., jvn)
```

% 创建 类型为 Vector{Int} 的 Julia 对象

```
A = Julia([2,1,3], 'Vector{Int}');
% 调用 Julia 函数需输入 Julia 对象
jv1 = jcall('TyBase.sort', A)
```

% M类型默认转换为 Julia 类型，转换后为矩阵

```
A = Julia([2,1,3]);
jv1 = jcall('TyBase.sort', A)
```

TyMLang.JuliaError:UndefKeywordError: UndefKeywordError: keyword argument `dims` not assigned

```
at /jcall.m:2:18, v2_call
at /jcall.m:2:18, jcall
at C:\Users\TR\Desktop\Untitled2.m:9:5, jcall
at C:\Users\TR\Desktop\Untitled2.m:9:5, Untitled2
```

```
jcall(__, '-kwargs', kws)
```

```
A = Julia([2,1,3]);
```

```
jv1 = jcall('TyBase.sort', A)
```

% 关键字参数为结构体，字段名为 Julia 函数中关键字的 key 值

```
jkwargs.dims = Julia(2, 'Int');
```

% '-kwargs' 标识关键字参数

```
value2 = jcall('TyBase.sort', A, '-kwargs', jkwargs)
```

% 运行结果

```
jv1 =
1x1 domain_proxy
 [1, 2, 3]
```


3.1 M 语言调用 Julia 函数-第三步: Julia 类型转为 M 类(fromJulia)

fromJulia 函数用法

% 将 Julia 类型转为 M 类型

```
o = fromJulia(jv)
```

```
o = fromJulia(jv, mtype)
```

`o = fromJulia(jv)`: 默认转换, `jv` 为 Julia 类型对象, 将 `jv` 转为 M 类型对象

`o = fromJulia(jv, mtype)`: 将 `jv` 转为类型为 `mtype` 的 M 类型对象, 此场景较为少用

% M 类型不需要区分标量、向量和数组, 可以直接转为 M 的对应类型的数组

```
jv1_1 = Julia(rand(3,1))
```

```
jcall('typeof', jv1_1)
```

```
o1_1 = fromJulia(jv1_1)
```

```
class(o1_1)
```

```
jv1_2 = Julia(1)
```

```
jcall('typeof', jv1_2)
```

```
o1_2 = fromJulia(jv1_2)
```

```
class(o1_2)
```

% 使用场景较少, 一般自动转换就足够使用
% 该用法用于有时 Julia 返回值为不确定, 可能为 Int、Float64、ComplexF64, 但是 M 代码中希望值为 double 类型

% Julia 的 Vector 转为 M 类型时转为行向量:

```
jv2_1 = Julia([1,2,3])
```

```
jcall('typeof', jv2_1)
```

```
o2_1 = fromJulia(jv2_1, 'double')
```

```
class(o2_1)
```

% Julia 与 MLang 数据类型对应关系

Float64 -> double

Float32 -> single

ComplexF64 -> complex

String -> string

% Char

UInt8 -> uint8

UInt16 -> uint16

UInt32 -> uint32

UInt64 -> uint64

Int8 -> int8

Int16 -> int16

Int32 -> int32

Int64 -> int64

% Struct(NamedTuple)

红色为暂不支持转换类型

3.1 M 语言调用 Julia 函数-示例: latcfilt 格型滤波器

latcfilt

晶格和晶格梯形滤波器实现

语法

```
[f,g] = latcfilt(k,x)
[f,g] = latcfilt(k,v,x)
[f,g] = latcfilt(k,1,x)
[f,g,zf] = latcfilt(__,"ic",zi)
[f,g,zf] = latcfilt(__,dim)
```

描述

[f, g] = latcfilt(k, x) 使用指定的 FIR 晶格系数对输入信号进行滤波, 并返回前向晶格滤波器结果和后向滤波结果。

例子

FIR 晶格滤波器

生成具有 512 个高斯白噪声样本的信号。

```
x = randn(512,1);
```

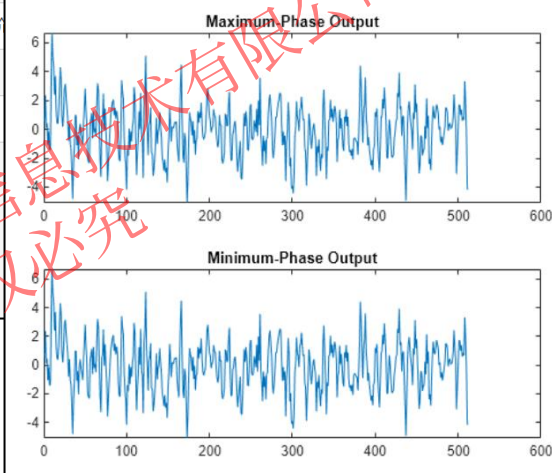
使用 FIR 晶格滤波器过滤数据。指定反射系数, 使晶格滤波器等效于三阶

```
[f,g] = latcfilt([1/2 1],x);
```

在单独的绘图中绘制晶格滤波器的最大相位和最小相位输出

```
subplot(2,1,1)
plot(f)
title('Maximum-Phase Output')
```

```
subplot(2,1,2)
plot(g)
title('Minimum-Phase Output')
```

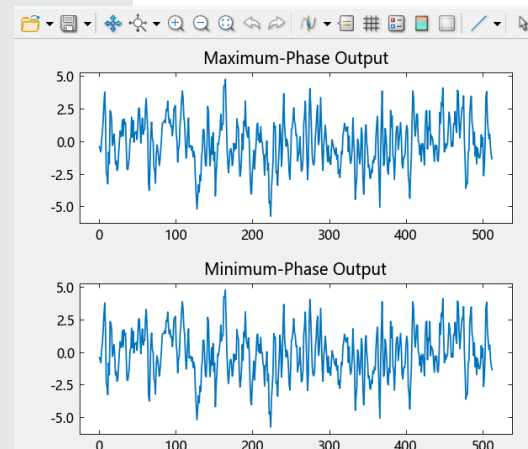


```
x = randn(512,1);
[f,g] = latcfilt([1/2 1],x); 现有MLang
subplot(2,1,1)
plot(f)
title('Maximum-Phase Output')
subplot(2,1,2)
plot(g)
title('Minimum-Phase Output') 代码见附件Test文件夹-tt文件夹-latciilt.m
```

tymlang.VarNotFound: 变量或函数 latcfilt 无法识别

```
%%
%原条件
x = randn(512,1);
%原函数
[f,g] = latcfilt([1/2 1],x);
%%
%调用实现
k=[1/2 1]
% 输入参数转为 Julia 类
jv_k = Julia(k);
jv_x = Julia(x)
% 根据 Syslab 帮助文档, 该函数有多个输出
jv = jcall('TySignalProcessing.latcfilt', jv_k, jv_x);
% 使用 jindex 将多个输出拆开
jv_1= jindex(jv, 1);
jv_2= jindex(jv, 2);
% 将输出参数转为 M 类型
f = fromJulia(jv_1);
g = fromJulia(jv_2);
%%
% 绘图
subplot(2,1,1)
plot(f)
title('Maximum-Phase Output')
subplot(2,1,2)
plot(g)
title('Minimum-Phase Output')
```

自行封装Julia的MLang



代码见附件Test文件夹-tt文件夹-latciilt_demo.m

3.1 M 语言调用 Julia 函数-未兼容原因及处理

未兼容原因	报错详情	示例	解决方案
函数未支持、 参数异常	tymlang.VarNotFound: 变量或函数 **无法识别/未定义 或者 TyMLang.ArgumentParserError: 期望参数 是**类型/输入参数不足	<pre>x = randn(512,1); [f,g] = latcfilt([1/2 1],x); subplot(2,1,1) plot(f) title('Maximum-Phase Output') subplot(2,1,2) plot(g) title('Minimum-Phase Output')</pre>	1. 将 Julia 函数封装为 M 函数 2. 联系同元工程师
JuliaError	TyMLang.JuliaError:Exception: 创建或更新**时出错，数组的形状 或大小错误	<pre>[XMesh, YMesh] = meshgrid(1:800, 301:600); ZMesh = zeros(size(XMesh)); CMesh = [800, 300]; surf(XMesh, YMesh, ZMesh, 'CData', CMesh, 'EdgeColor', 'none');</pre>	Julia侧的报错，原因是Julia函数 有这种用法，但是使用有误，这 种用法在M软件里面一般也会报 错
未找到xx	tymlang.PropertyNotFound: 属性** 未在 GraphicObject 中找到	<pre>% ax ax = gca; ax.XTick=[];</pre>	目前不支持这种.索引 替代方案: set(ax, 'XTick', [])

3.2 M直接调用Python

支持 **M 直接调用Python**功能：提供pyrun、pyrunfile、pyargs等核心函数

```
py_test.m x
1 % 从 MATLAB 中运行 Python 语句
2 pyrun("import localModule")
3 out = pyrun(["m = localModule.mvar", "localModule.myFunc()"], "m")
4
5 % 从 MATLAB 运行 Python 脚本文件
6 res = pyrunfile("addac.py", "z", "x", 3, "y", 2)
7
8 % 为 Python 函数创建关键字参数
9 py.complex(pyargs('real', 1, 'imag', 2))
10
11

localModule.py x
1 def myFunc():
2     print('myFunc executed')
3     mvar = 3
4
5

addac.py x
1 def add(a, c):
2     b = a + c
3     return b
4
5 z = add(x, y)
```

输出 调试控制台 终端

```
py_test
myFunc executed
out =
PyInstance:
  PyObject 3
res =
PyInstance:
  PyObject 5
ans =
PyInstance:
  PyObject (1+2j)
```

%从 MATLAB 中运行 Python 语句
pyrun("import localModule")
out = pyrun(["m= localModule.mvar", "localModule.myFunc()"], "m")
%从 MATLAB 运行 Python 脚本文件
res = pyrunfile("addac.py", "z", "x", 3, "y", 2)
%为 Python 函数创建关键字参数
py.complex(pyargs('real', 1, 'imag', 2))

```
def add(a, c):
    b = a + c
    return b

def myFunc():
    print('myFunc executed')
mvar = 3

z = add(x, y)
```

```
main_test
myFunc executed
out =
PyInstance:
  PyObject 3
res =
PyInstance:
  PyObject 5
ans =
PyInstance:
  PyObject (1+2j)
```

3.3 Julia语言调用M函数

支持在 Julia 中执行 M 代码、操作 M 语言的对象、类型转换以及调用 M 语言的函数

函数	语法	说明
eval_mcode!	<code>eval_mcode!(code)</code>	在 Julia 中直接运行 M 代码, 返回值为 <code>nothing</code>
to_mlang	<code>to_mlang(x)</code>	将一个 Julia 对象转换为 <code>MxArray</code> 类型, 支持数值类型和对应的数组, 以及标量的 <code>String</code> 类型
from_mlang	<code>from_mlang(pm::MxArray)</code> <code>from_mlang(T::Type, pm::MxArray)</code>	- 将 <code>MxArray</code> 类型转换对应的 Julia 类型, 支持数值类型和对应的数组类型, 以及标量的字符串, 其他类型仍返回 <code>MxArray</code> - 将 <code>MxArray</code> 类型转换为 <code>T</code> 类型, 支持数值类型和对应的数组类型, 以及标量的 <code>String</code> 类型, 类型不匹配时返回 <code>nothing</code>
mexCallMLang	<code>mexCallMLang(funName::String, nargsout::Integer, args::Vector{MxArray})</code>	<code>funName</code> 为调用的 M 函数, <code>nargsout</code> 为 M 函数的返回参数个数, <code>args</code> 为调用的 M 函数的输入参数, 返回值为 <code>Vector{MxArray}</code>
mexGetVar	<code>mexGetVar(varName::String)</code>	获取 <code>MLang REPL</code> 中的变量, 返回值为 <code>MxArray</code>
mxget_cell	<code>mxget_cell(pm::MxArray, i)</code>	获取元胞数组 <code>pm</code> 的第 <code>i</code> 个元素, 返回值为 <code>MxArray</code> 类型
mxset_cell	<code>mxset_cell(pm::MxArray, i, value::MxArray)</code>	将元胞数组 <code>pm</code> 的第 <code>i</code> 个元素设置为 <code>value</code>
mxget_field	<code>mxget_field(pm::MxArray, i, fieldname)</code>	获取结构体 <code>pm</code> 的第 <code>i</code> 个元素的 <code>fieldname</code> 字段, 返回值为 <code>MxArray</code> 类型
mxset_field	<code>mxset_field(pm::MxArray, i, fieldname, value::MxArray)</code>	将结构体 <code>pm</code> 的第 <code>i</code> 个元素的 <code>fieldname</code> 字段设置为 <code>value</code>
其他 API	详见帮助文档	

3.3 Julia语言调用M函数

Julia 调用 M 兼容新增函数接口，支持数值类型变量在 Julia 工作区和 M 工作区之间转换

Syslab使用手册

全局搜索

目录

« 使用 Syslab

« 多语言编程环境

« M 语言计算环境

快速入门

基本语法

支持的 M 函数列表

M 代码混淆处理

高阶 - M 语言调用 Julia 函数

高阶 - Julia 语言调用 M 函数

eval_mcode!

to_mlang

from_mlang

mexCallMLang

mexGetVar

mexSetVar

@import_from_mlang

@export_to_mlang

mxget_cell

mxset_cell

mxget_field

mxset_field

其他 API

示例

局限性

文档

@import_from_mlang

- 语法

```
@import_from_mlang varName
```
- 说明

@import_from_mlang varName, 导入 MLang REPL 中的变量, 并自动转换为 Julia 变量, 目前仅支持数值类型或标量字符串类型变量的自动转换。
- 示例

▼ 示例 1: 在 Julia 中导入 MLang REPL 中的变量

```
% 在 MLang REPL 中定义变量
bar = [6 5 4; 3 2 1];

# 在 Julia REPL 获取 MLang REPL 的变量 foo
using TyMLangCore
TyMLangCore.@import_from_mlang bar;

bar

2x3 Matrix{Float64}:
6.0  5.0  4.0
3.0  2.0  1.0
```

@export_to_mlang

兼容局限性

分类	详情	示例
Win7 操作系统的局限性	• 日期和文件系统API兼容的局限性 • 在 Win7 环境下，解析器不是并行的，解析较多文件耗时更长 • 在 Win7 环境下，暂不支持 tar 命令	• 对于dir返回的文件/文件夹项，date一律为 1900-01-01，datenum一律为 0 • now函数等时间相关的函数无法使用
数据类型的局限性	• M 语言计算环境暂不支持部分 MATLAB 数据类型，如：datetime, categorical 等 • M 语言计算环境不支持导入由部分经 MATLAB 保存至 MAT 文件的数据类型，如：ss, tf, zpk 等	• datetime 是支持单独使用，但是不支持传入到函数里面
功能的局限性	• M 语言计算环境暂不支持部分 MATLAB 功能，如：并行计算、APP 构建等 • M 语言计算环境在脚本文件中使用 input 函数时，第一个 input 需要输入两次	• input函数的输入属于Julia的调用问题

@苏州同元软控信息技术有限公司版权所有，侵权必究

Thanks.

建立知识规范，营造协同生态
积累工业模型，发展可控平台
融入工业创新，共创先进软件

